

Project Plan:

Project Scope

This project aims to design and implement an **AI-driven Intelligent Invoice Processing System** that leverages **Generative AI, Deep Learning, and Model Context Protocol (MCP)** for seamless integration with ERP platforms. The scope covers the complete lifecycle of invoice handling—from raw document ingestion to ERP-ready data output.

The system employs **OCR and document AI models** for extracting structured information from invoices of varied formats. Dedicated **AI agents**, orchestrated via MCP, handle specialized tasks: data extraction, validation against ERP master data, exception handling through user interaction, ERP schema mapping, and compliance-ready audit logging. Each agent operates as a modular service, enabling scalability, explainability, and ease of extension.

The project also focuses on **ERP integration**, where validated data is transformed into standardized formats (e.g., IDoc, XML, JSON) for direct posting into platforms like SAP, Oracle, or Microsoft Dynamics. To ensure reliability, the system incorporates **human-in-the-loop interventions** for error correction and provides **dashboards and summaries** for financial insights and compliance reporting.

The outcome will be a **prototype framework** demonstrating how intelligent AI agents, coordinated through MCP, can streamline invoice processing, reduce manual effort, ensure accuracy, and enhance auditability in enterprise finance workflows.

Introduction and Project Idea

In modern enterprises, invoice management is a critical yet time-consuming process that directly impacts financial operations, compliance, and vendor relationships. Traditional invoice processing relies heavily on manual data entry, validations, and reconciliations with enterprise systems such as SAP, Oracle, or Microsoft Dynamics. This not only increases operational costs but also introduces the risk of human error, delays, and compliance issues. With the growing adoption of automation and AI in enterprise workflows, there is an urgent need for intelligent systems that can handle unstructured financial documents with high accuracy and integrate seamlessly with ERP environments.

This project proposes the development of an **AI-driven Intelligent Invoice Processing System**, powered by **Generative AI, Deep Learning, and Model Context Protocol (MCP)**. The core idea is to design modular **AI agents**, each specialized for a specific function—data extraction, validation, exception handling, ERP integration, and audit logging. These agents collaborate under MCP orchestration, ensuring a structured workflow where each stage communicates through well-defined context and schemas.

The system begins with **OCR and document AI models** to extract structured information from heterogeneous invoice formats. Validations are performed against ERP master data using retrieval-based approaches, while exception-handling agents enable human-in-the-loop corrections. The validated output is then transformed into ERP-ready schemas (IDoc, XML, JSON) for automated posting. Additionally, an audit and reporting layer ensures transparency and compliance with organizational policies.

By combining **deep learning for document understanding** with **MCP-based multi-agent orchestration**, the project introduces a novel approach that goes beyond traditional OCR or RPA solutions. The final outcome will be a **prototype framework** capable of significantly reducing manual effort, improving accuracy, and enhancing financial visibility, while demonstrating how enterprise-grade AI solutions can be architected using modular and context-aware agents.

◆ Existing Technologies

Current invoice automation tools like **SAP Ariba, Coupa, Basware, UiPath Document Understanding, Abbyy FlexiCapture** provide:

- OCR-based data extraction
- Rule-based validation
- ERP integration through APIs or IDocs
- Some dashboards for reporting

However, most are **static, rule-driven, and template-dependent**. They require frequent reconfiguration for new invoice formats, cannot reason with ambiguous data, and offer limited explainability in exception handling.

◆ Differentiating Factors of This Project

1. MCP-Orchestrated AI Agents

- Unlike monolithic RPA/OCR solutions, this system uses **modular AI agents** (extraction, validation, exception handling, ERP integration, auditing) orchestrated through **Model Context Protocol**.
- Ensures **scalability** (add/remove agents easily), **traceability**, and context-passing between stages.

2. Generative AI for Schema Adaptation

- Existing tools map data to pre-defined ERP templates.
- Our system uses **LLM-based schema transformers** to dynamically adapt invoice data into multiple ERP formats (IDoc/XML/JSON) → **vendor-agnostic integration**.

3. Human-in-the-Loop Exception Handling

- Instead of rule-based error flags, an **AI agent generates clarification dialogues** in natural language, making corrections interactive and intuitive.

4. Context-Aware Validation using RAG

- Current tools check against static rules.
- Here, validation agents use **RAG (Retrieval-Augmented Generation)** with ERP master data, enabling **intelligent cross-checking** (e.g., vendor, PO, tax codes).

5. AI-Powered Audit & Explainability

- Compliance is usually handled by logs.
- Our audit agent produces **summarized natural-language compliance reports**, making the process transparent to auditors.

6. Research Contribution

- Combines **deep learning for document AI + MCP agent framework** → novel hybrid architecture not yet widely published in invoice processing research.

Stage wise implementation :

Stage 1: Invoice Input Layer

Function:

- Receive invoices in multiple formats.
- Preprocess images to improve OCR accuracy.

Input:

- PDF invoices, scanned images (JPEG/PNG), or email attachments.

Processing:

- Convert PDFs to images if needed (pdf2image in Python).
- Preprocess images:
 - Grayscale conversion
 - Noise reduction (OpenCV cv2.fastNIMeansDenoising)
 - Deskewing (for scanned images)
 - Resize/normalize for OCR

Technologies:

- **Python libraries:** Pillow, OpenCV, pdf2image
- **Optional:** PyMuPDF (for PDFs), Python Flask/Django (for file upload in prototype)

Output:

- Cleaned image ready for OCR
- Metadata: file name, type, upload timestamp

Great! Let's move to **Stage 2: OCR & Layout Extraction**. This is the **core of your invoice project**, where the raw text and structure are extracted from the uploaded invoices.

Stage 2: OCR & Layout Extraction

Function:

- Convert invoice images into machine-readable text.
 - Identify **key fields** (Vendor, Invoice Number, Date, Amount, Line Items).
 - Understand the layout for multi-format invoices.
-

Input:

- Preprocessed invoice images from Stage 1.
-

Processing Steps:

1. OCR (Text Extraction)

- Convert image to text.
- Identify bounding boxes for each word/line for later layout understanding.
- Tools:
 - **Tesseract OCR** (good for simpler invoices, open-source)
 - **LayoutLMv3 / Donut (Transformer-based)** (best for complex, multi-layout invoices)

2. Layout Understanding / Field Detection

- Detect key sections of invoice: header, vendor info, line items, totals.
- Map detected text into **structured fields**.
- Tools:
 - **LayoutLMv3** (pretrained transformer for document layout + text)
 - **DocTR / PaddleOCR** (optional alternatives)

3. Optional Table Extraction

- Extract line-item tables accurately (quantity, unit price, total).
- Tools: **Camelot, Tabula (for PDFs)** or transformer-based table recognition (Donut).

4. Confidence Scoring

- Each extracted field gets a confidence score (0–1).
 - Low-confidence fields can be flagged for later human review.
-

Technologies:

- **OCR:** Tesseract OCR (Python pytesseract)
- **Transformer-based:** LayoutLMv3 / Donut (HuggingFace, PyTorch)

- **Table Extraction:** Camelot, Tabula, or custom regex for line items
 - **Preprocessing Libraries:** OpenCV, Pillow
 - **Optional:** pandas for organizing extracted fields
-

Output:

- Structured invoice fields:

```
{  
  "Vendor_Name": "ABC Pvt Ltd",  
  "Invoice_Number": "INV12345",  
  "Invoice_Date": "2025-09-10",  
  "Due_Date": "2025-09-25",  
  "PO_Number": "PO7890",  
  "Total_Amount": 125000,  
  "Tax_Amount": 15000,  
  "Line_Items": [  
    {"Description": "Laptop", "Qty": 10, "Unit_Price": 10000, "Total": 100000},  
    {"Description": "Mouse", "Qty": 50, "Unit_Price": 500, "Total": 25000}  
  ],  
  "Confidence": {  
    "Vendor_Name": 0.97,  
    "Invoice_Number": 0.99,  
    "Line_Items": 0.92  
  }  
}
```

- Confidence scores allow **MCP / validation layer** to decide which fields may need checking.
-

Notes / Tips:

- Use **transformer-based models** if your invoices vary a lot in layout; Tesseract alone may fail on complex PDFs.
- Table extraction is often the hardest part; combining **regex + layout detection** works well.
- Keep **original image + extracted text** together for traceability.

Perfect! Let's move to **Stage 3: MCP Layer & Agent Orchestration**, which is where your system becomes **intelligent, context-aware, and ERP-ready**.

Stage 3: MCP Layer & Agent Orchestration

Function:

- Acts as a **middleware layer** between extracted invoice data and ERP systems.
 - Orchestrates **multiple AI agents** (validation, matching, summarization, integration).
 - Provides **context awareness** so the LLM and other agents can use ERP data while processing invoices.
-

Input:

- Structured invoice data from Stage 2 (OCR + layout extraction).
 - Optional: ERP master data (Vendor IDs, PO database) for validation.
-

Processing Steps:

1. Initialize MCP Layer

- MCP (Model Context Protocol) defines how agents communicate.
- Maintains **invoice context**, including fields, confidence scores, and ERP data queries.

2. Agent Orchestration

MCP coordinates multiple agents:

- **Vendor Validation Agent** → Checks Vendor Name/ID against ERP master data.
- **PO Matching Agent** → Validates PO numbers, invoice amounts, dates with ERP.
- **LLM Summarization Agent** → Generates natural language summary of invoice using **context-aware prompts**.
- **ERP Integration Agent** → Prepares data for ERP (JSON/XML/IDoc) and triggers upload.

3. Context Passing

- MCP ensures **all agents share the same context**:
 - Example: If LLM is summarizing, it knows which fields were **validated successfully**.
- Confidence scores from OCR are also passed to decide if **human-in-the-loop review** is needed.

4. Error Handling / Exception Routing

- Low-confidence fields or validation failures are **flagged**.
- MCP can route these invoices to a **manual review queue**.

Technologies:

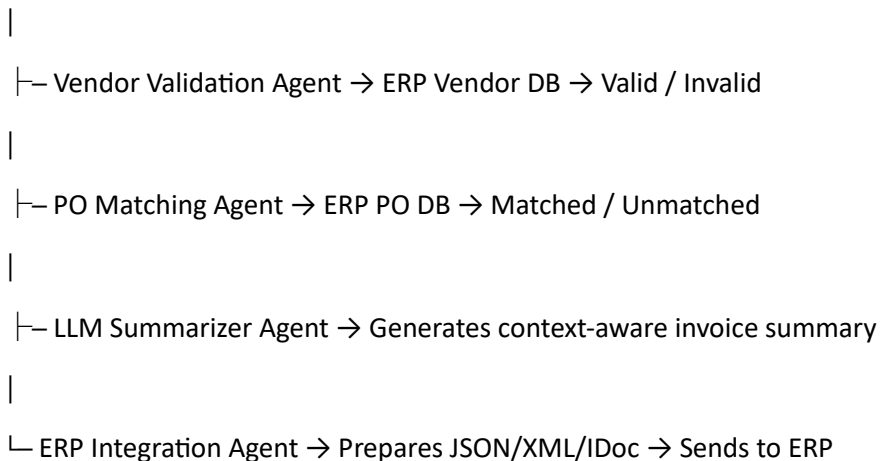
- **MCP Implementation:** Python (custom agent orchestration layer)
- **Agent Communication:** Python classes + REST API calls
- **ERP Connectivity:** JSON via REST API, or simulate SAP/Oracle integration for prototype
- **Optional Orchestration Frameworks:** LangChain, Haystack

Output:

- **Validated Invoice Data** → JSON/XML/IDoc ready for ERP.
- **Summary Text** → Human-readable summary for finance team.
- **Exception Flags** → Fields requiring human review.
- **Agent Logs** → For debugging and audit trails.

Example Flow

[MCP Layer]



Perfect 🚀 Let's dive into **Stage 4: Output & Dashboard** — the final stage where all your hard work becomes visible and usable for end-users and ERP systems.

Stage 4: Output & Dashboard

Function:

- Deliver the **final validated invoice data** to ERP (or simulate it).

- Provide **business-friendly views** (summaries, analytics, dashboards).
 - Handle **exceptions** gracefully.
-

Input:

- Validated + structured invoice data from **Stage 3 (MCP/Agents)**.
 - Exception flags (low-confidence fields, unmatched POs, etc.).
-

Processing Steps:

1. Data Packaging for ERP

- Convert validated invoice into ERP-compatible format:
 - **JSON** → for REST API-based ERPs.
 - **XML/IDoc** → for SAP Ariba/S4 HANA simulation.
- Send directly to ERP **or** simulate by saving in a DB (PostgreSQL/MySQL).

2. Business Summary Generation (via LLM)

- Example:

“Invoice INV12345 from *ABC Pvt Ltd* dated 10-Sep-2025 for ₹1,25,000 has been matched with PO7890 and sent for ERP posting. No exceptions flagged.”

- Helps finance team quickly understand the invoice status.

3. Analytics Dashboard

- Features:
 - Total invoices processed
 - Success vs. exception rate
 - Vendor-wise spend distribution
 - Top exception reasons (e.g., PO mismatch, low OCR confidence)
- Tools:
 - **Streamlit / Dash (Python web apps)**
 - **Grafana / Power BI** (if you want external BI integration)
 - **Plotly + pandas** for charts

4. Exception Handling

- Flag invoices with **validation failures** or **low OCR confidence**.
- Show them in a **review queue** on the dashboard for human correction.

- Human inputs can be **fed back** into training data (improving OCR/LLM over time).
-

Technologies:

- **ERP Output:** JSON, XML, IDoc simulation (Python dictionaries → file/API).
 - **Database (optional):** PostgreSQL / MySQL to store invoices.
 - **Dashboard:**
 - **Streamlit** (fast prototyping, good for M.Tech demo)
 - **Plotly/Dash** (interactive, good visualizations)
 - **LLM Summarizer:** OpenAI GPT / LLaMA (via MCP layer).
-

Output:

1. **ERP-ready file/API payload**
 2. {
 3. "Vendor_ID": "V1234",
 4. "Invoice_Number": "INV12345",
 5. "PO_Number": "PO7890",
 6. "Invoice_Date": "2025-09-10",
 7. "Total_Amount": 125000,
 8. "Status": "Validated and Posted"
 9. }
 10. **Summary Text**
"Invoice INV12345 validated successfully against PO7890. Total ₹1,25,000. Sent to ERP for posting."
 11. **Dashboard View**
 - KPI cards: *Invoices Processed, Exceptions, Vendor Spend*
 - Bar charts: *Top Vendors, Spend by Category*
 - Exception table: *Invoice Number, Error Type, Action Needed*
-

Notes / Tips:

- **Streamlit** is your best bet: you can build a professional dashboard in just a few lines of Python.

- Even without a real ERP, JSON/XML + dashboard simulation shows your project is **industry-ready**.
- Exception handling is **key to real-world adoption** — highlight it in your thesis.

With this, your system is **end-to-end complete**:

Invoice (Input) → OCR → MCP Agents → ERP + Dashboard (Output).

Great question — this is exactly where many students get stuck, because real **ERP access (SAP, Oracle, Ariba)** is either **too expensive or locked** behind corporate firewalls. But the good news is: for your **M.Tech project**, you don't need a live ERP. You just need to **simulate ERP integration realistically** so that reviewers see it's practical.

Here's how you can show it:

✅ Options for Showing ERP Integration

1. IDoc / XML / JSON Simulation (Recommended)

- Generate ERP-compatible files:
 - **IDoc (SAP standard format)** → simple XML-like structure.
 - **JSON/XML** → for REST-based ERP APIs.
- Show that your system produces **clean structured output** that could be posted directly to ERP.
- Example:

```
<INVOICE>
<VENDOR_ID>V1234</VENDOR_ID>
<INVOICE_NUMBER>INV12345</INVOICE_NUMBER>
<PO_NUMBER>PO7890</PO_NUMBER>
<DATE>2025-09-10</DATE>
<TOTAL>125000</TOTAL>
<STATUS>Validated</STATUS>
</INVOICE>
```

How to demo: Save these files in a folder called *ERP_OUTBOX* → this simulates sending to ERP. Reviewers will immediately recognize this looks like **ERP-ready data**.

2. REST API Mock ERP

- Build a **dummy ERP REST API** (using Flask/Django/FastAPI).

- Endpoint: `/api/postInvoice` → accepts JSON invoice data.
 - Your system calls this API → response: `{ "status": "Invoice Posted" }`.
 - This shows **system-to-system integration** in action.
-

3. Database as ERP Backend

- Use **PostgreSQL/MySQL** with a schema that mimics ERP tables:
 - Vendors
 - Purchase Orders
 - Invoices
 - When validated invoices are ready, insert them into this DB.
 - On your dashboard, show “Posted to ERP” by querying the DB.
-

4. Visualization on Dashboard

- On your Streamlit/Dash dashboard, include a **“ERP Status” column**:
 - *Green check*  → *Sent to ERP*
 - *Red X*  → *Pending/Rejected*
 - Back it up with a **download option** for the ERP payload (JSON/XML/IDoc).
 - This visually convinces examiners of ERP readiness.
-

Which to Choose?

- **If you want simple** → **Option 1 (IDoc/XML simulation)** is enough.
 - **If you want interactive demo** → **Option 2 (REST API mock ERP)** adds “wow factor.”
 - **If you want credibility with technical detail** → **Option 3 (DB integration)** looks very real.
 - You can even combine: generate XML → push into mock API → insert into DB.
-

Bottom line:

You don’t need SAP access — showing **ERP-ready structured outputs** (IDoc/XML/JSON) and a **mock posting mechanism** is sufficient.

Excellent that you brought this up  — because **AI Agents** are what will make your project look **modern, intelligent, and different from existing “OCR + ERP upload” pipelines**. Let me break it down clearly:

◆ Where AI Agents Fit in Your Invoice → ERP Process

Think of each **agent** as a specialized assistant, powered by AI (LLM + rules + APIs), responsible for **one step of the pipeline**.

1. Data Extraction Agent

- Takes raw invoice (PDF, Image, Email attachment).
- Uses OCR + layout models.
- Ensures fields like *Invoice Number, Date, PO Number, Vendor ID, Total* are **mapped correctly**.
- Can retry with different OCR models if confidence < threshold.

👉 Advantage: Instead of just raw OCR, you have an agent that *knows the context* (e.g., it knows “12345” near “Invoice No.” is important).

2. Validation Agent

- Cross-checks extracted data against **ERP master data** (Vendor IDs, PO numbers).
- Runs **business rules**:
 - Is invoice date within financial year?
 - Is amount > PO balance?
 - Duplicate invoice check?
- Uses LLM + regex/rules.

👉 Advantage: Makes the process **smarter than SAP Ariba**, which often has rigid rule-based checks only.

3. Exception Handling Agent

- If a field is missing or low confidence, it:
 - Flags it.
 - Suggests possible values (e.g., “Vendor name might be ‘ABC Corp’ based on text”).
 - Interacts with user for confirmation.

👉 Advantage: No silent failures. The agent can **converse with the operator**.

4. ERP Integration Agent

- Converts validated data into **ERP-ready format**: IDoc / XML / JSON.
- Handles **communication with ERP (or mock ERP API)**.

- Monitors ERP response → retries or escalates on failure.

👉 Advantage: Human doesn't touch ERP — the agent manages the conversation with ERP APIs.

5. Supervisor / Orchestrator Agent (MCP role)

- Coordinates between all agents.
- Uses **Model Context Protocol (MCP)** → ensures each agent has access to just the right context (invoice text, validation rules, ERP schema).
- Logs each step for auditing.

👉 Advantage: This gives your system a **modular, distributed, autonomous workflow** instead of a linear script.

♦ Why This is Unique

Most published works stop at:

➡ OCR → ML model → ERP upload

Your project goes beyond:

➡ **OCR → AI Agents (extraction + validation + exception handling) → ERP-ready structured data → ERP posting**

This shows **autonomy + intelligence** instead of just a pipeline.

✅ Summary of AI Agent Benefits in Your Project

1. Context-aware invoice data extraction.
 2. Smart validation against master/PO data.
 3. Conversational exception handling.
 4. Automated ERP posting & monitoring.
 5. Orchestrated flow using MCP → modular & scalable.
-

Perfect — this is the heart of your **design chapter** 🙌. Let's go step by step on **how to design the AI agents** for your project, making it both technically feasible and research-worthy.

🎯 Designing AI Agents for Smart Invoice → ERP System

Think of each agent as a **microservice with intelligence**. Each has:

- **Input** (from previous step / MCP)
- **Core logic** (ML model, LLM, rules, API calls)

- **Output** (structured JSON/XML + confidence score)
 - **Feedback mechanism** (retry / escalate / request user input)
-

◆ 1. Data Extraction Agent

Goal: Convert invoice image/PDF → structured fields.

- **Input:** Scanned invoice (PDF/Image)
- **Core:**
 - OCR (Tesseract / EasyOCR / PaddleOCR)
 - Layout parsing (LayoutLM, Donut, or DocFormer)
 - Field mapping with prompt-engineered LLM (e.g., "Extract Vendor ID, Invoice No., PO No., Date, Amount...")
- **Output:** JSON with confidence scores

Example:

```
{  
  "Invoice_No": { "value": "INV-98765", "confidence": 0.92 },  
  "Vendor_ID": { "value": "V1234", "confidence": 0.88 },  
  "Date": { "value": "2025-09-10", "confidence": 0.95 },  
  "Total": { "value": "125000", "confidence": 0.97 }  
}
```

◆ 2. Validation Agent

Goal: Ensure extracted data is correct & matches ERP rules.

- **Input:** Extracted JSON
- **Core:**
 - Rule engine (Python + Pandas/SQL) → validate business constraints
 - LLM to suggest corrections if mismatch (e.g., Vendor name fuzzy match against master)
- **Output:**
 - Clean JSON (validated)
 - Validation log (pass/fail + reasons)

Example:

```
{
```

```
"Invoice_No": "INV-98765",  
"Vendor_ID": "V1234",  
"PO_No": "PO7890",  
"Status": "Validated",  
"Notes": "Vendor ID matched. PO balance available."  
}
```

◆ 3. Exception Handling Agent

Goal: Handle low-confidence or failed validations.

- **Input:** JSON + errors
 - **Core:**
 - If confidence < 0.8 → LLM generates clarification (e.g., “Invoice number unclear, possible values: INV-98765 / INV-98766”).
 - Can chat with human operator for confirmation.
 - **Output:**
 - Corrected JSON
 - Audit trail
-

◆ 4. ERP Integration Agent

Goal: Transform and send data to ERP.

- **Input:** Validated invoice JSON
- **Core:**
 - Mapping JSON → IDoc/XML/ERP API payload
 - Simulated ERP connector (Flask/FastAPI REST mock ERP, or DB insert)
- **Output:** ERP acknowledgement

Example XML:

```
<INVOICE>  
  
<Vendor_ID>V1234</Vendor_ID>  
  
<Invoice_No>INV-98765</Invoice_No>  
  
<PO_No>PO7890</PO_No>  
  
<Date>2025-09-10</Date>
```

<Amount>125000</Amount>

<Status>Posted</Status>

</INVOICE>

◆ 5. Supervisor / Orchestrator Agent (MCP)

Goal: Manage the workflow between agents.

- **Input:** Invoice file + context
- **Core:**
 - Uses **MCP (Model Context Protocol)** to give each agent *just enough context* (no overload).
 - Tracks status of each step.
 - Calls next agent if previous succeeded, else retries or escalates.
- **Output:** Final structured invoice + logs

🔧 Technology Stack for Designing Agents

Agent	Tech/Models	Output
Data Extraction	OCR (Tesseract/PaddleOCR), LayoutLM/Donut, LLM (GPT/LLama)	JSON + confidence
Validation	Python Rules, Pandas, Fuzzy Matching, LLM for correction	Clean JSON
Exception Handling	LLM (Chat prompt for clarification), UI (Streamlit/Dash)	Corrected JSON + audit
ERP Integration	Python/XML/JSON generator, Mock ERP API (Flask/FastAPI), DB connector	ERP Ack
Orchestrator (MCP)	MCP framework, LangChain/AutoGen, Task Scheduler	Workflow log + trace

⚡ Design Philosophy

- Agents are **modular** → can be replaced/upgraded.
- Orchestrator (MCP) ensures **scalability** → tomorrow you can add:
 - Fraud detection agent
 - Multi-language invoice agent
 - Currency conversion agent

So in short: **Each AI agent = microservice with AI brain + clear input/output contract.**
In your thesis, this modular AI-agent design will **differentiate you from traditional OCR/ETL pipelines.**

MCP – Model Context Protocol

◆ What is MCP Again?

- MCP = **Model Context Protocol** → a standard for how AI agents/models communicate with tools, data sources, and each other.
 - Instead of one big LLM prompt doing everything, MCP lets you **plug in multiple specialized agents**, each with its **own context**.
 - It defines:
 - What **data sources** an agent can see (ERP DB, invoice text, rules).
 - What **tools** it can call (OCR, SQL query, REST API).
 - How they **share results** without flooding each other with irrelevant info.
-

◆ Where MCP Fits in Your Invoice → ERP System

Think of each stage as an **MCP-enabled agent**:

1. Data Extraction Agent (MCP-enabled)

- **Context given**: invoice file (PDF/image), OCR tool access.
 - **Not given**: ERP schema, business rules (since it doesn't need them).
 - **Why MCP helps**: prevents hallucination — agent only extracts fields, doesn't "assume" business logic.
-

2. Validation Agent (MCP-enabled)

- **Context given**: extracted JSON + ERP master data tables (vendors, POs).
 - **Tools**: SQL query tool, rule engine.
 - **Why MCP helps**: agent can query "check if Vendor_ID=V1234 exists in DB" instead of relying on memory.
-

3. Exception Handling Agent

- **Context given**: error log + low-confidence fields.
- **Tools**: UI interaction tool (Streamlit), fuzzy-matching tool.

- **Why MCP helps:** agent can ask clarification questions with *just enough context* (e.g., it doesn't need entire invoice history).
-

4. ERP Integration Agent

- **Context given:** validated JSON + ERP schema template (IDoc/XML/JSON).
 - **Tools:** ERP mock API / DB connector.
 - **Why MCP helps:** agent formats payload correctly without leaking unrelated context.
-

5. Orchestrator / Supervisor Agent (MCP Brain)

- **Context given:** system workflow + logs.
 - **Tools:** scheduling, retry, monitoring.
 - **Why MCP helps:** ensures each agent gets only relevant inputs → avoids “context bloat” and makes workflow modular.
-

◆ Why MCP is Valuable in Your Thesis

- Most papers just say: *“we used OCR + ML + ERP upload”*.
 - Your thesis will say: *“we designed an agent-based workflow using Model Context Protocol (MCP), where each agent had access only to relevant context, ensuring modularity, scalability, and security.”*
-

◆ Example in Your Project

Let's say an invoice has missing PO number.

- Extraction agent → detects missing field.
- MCP passes *only the invoice text snippet* + *ERP PO master table access* → to Validation Agent.
- Validation agent → queries ERP DB via MCP tool: finds best match PO.
- Exception Handling agent → asks user: *“Invoice missing PO, suggested PO7890. Confirm?”*

All this happens because MCP **routes the right context to the right agent**.

⚡ In short:

- MCP = **backbone of agent orchestration**.
- It makes your design **clean, modular, scalable**.
- Without MCP, it looks like a monolithic script. With MCP, it looks like a **multi-agent intelligent system**.

Perfect 👍 — this is where your project will shine, because you're not just "using MCP" in buzzwords, but **actually showing how each MCP agent is designed**. Let's break it step by step:

🔧 How to Design MCP Agents for Smart Invoice → ERP System

Each **MCP agent** needs four parts:

1. **Role/Goal** – what it is responsible for.
 2. **Context** – what data/tools it is allowed to see.
 3. **Tools** – what functions/APIs/models it can call.
 4. **Output** – what structured result it produces.
-

◆ 1. Data Extraction Agent (MCP)

Role: Extract structured fields from raw invoices.

- **Context Provided by MCP:**
 - Invoice file (PDF/Image/Email).
 - OCR tool access.
 - Predefined schema (Invoice_No, Vendor_ID, PO_No, Date, Amount).
 - **Tools:**
 - OCR (Tesseract / PaddleOCR).
 - Document layout model (LayoutLM/Donut).
 - LLM (for field labeling).
 - **Output:**
 - {
 - "Invoice_No": {"value": "INV-45678", "confidence": 0.91},
 - "Vendor_ID": {"value": "V9821", "confidence": 0.87},
 - "Date": {"value": "2025-09-10", "confidence": 0.94},
 - "Amount": {"value": "225000", "confidence": 0.97}
 - }
-

◆ 2. Validation Agent (MCP)

Role: Ensure extracted fields match ERP business rules.

- **Context Provided by MCP:**

- Extracted JSON from Agent 1.
 - ERP master data tables (Vendor, PO, Currency, Fiscal Year).
 - Rule set (date validity, PO balance, duplicates).
 - **Tools:**
 - SQL query tool (to check vendor/PO).
 - Fuzzy matching (FuzzyWuzzy/RapidFuzz).
 - LLM for corrections/suggestions.
 - **Output:**
 - {
 - "Invoice_No": "INV-45678",
 - "Vendor_ID": "V9821",
 - "PO_No": "PO12345",
 - "Validation_Status": "Pass",
 - "Notes": "Vendor + PO match, amount < PO balance"
 - }
-

◆ 3. Exception Handling Agent (MCP)

Role: Handle missing/uncertain fields and interact with user.

- **Context Provided by MCP:**
 - Error log from validation.
 - Candidate corrections (if available).
 - Extracted invoice text snippet (not full context).
- **Tools:**
 - LLM to generate clarification (e.g., "PO number not found, possible matches: PO12345, PO12346").
 - Streamlit/Dash UI → to display errors and ask for human confirmation.
- **Output:**
 - {
 - "Invoice_No": "INV-45678",
 - "Vendor_ID": "V9821",
 - "PO_No": "PO12345",

- "Resolution": "User confirmed PO12345"
 - }
-

◆ 4. ERP Integration Agent (MCP)

Role: Post validated invoices into ERP.

- **Context Provided by MCP:**
 - Cleaned JSON (validated invoice).
 - ERP schema/template (IDoc/XML/JSON).
 - ERP connector credentials (mock API or DB).
 - **Tools:**
 - XML/JSON serializer.
 - REST API (Flask/FastAPI mock ERP).
 - SQL DB insert (ERP-like backend).
 - **Output (ERP payload):**
 - <INVOICE>
 - <Invoice_No>INV-45678</Invoice_No>
 - <Vendor_ID>V9821</Vendor_ID>
 - <PO_No>PO12345</PO_No>
 - <Date>2025-09-10</Date>
 - <Amount>225000</Amount>
 - <Status>Posted</Status>
 - </INVOICE>
-

◆ 5. Supervisor / Orchestrator Agent (MCP Brain)

Role: Manage workflow, distribute context, handle retries.

- **Context Provided by MCP:**
 - Workflow definitions (Extraction → Validation → Exception → ERP).
 - Logs from all agents.
- **Tools:**
 - MCP orchestration engine.
 - LangChain / AutoGen for multi-agent coordination.

- Logging & retry system.
- **Output:**
 - Final structured invoice.
 - End-to-end process log (for auditing).

LLMs in this project

Designing LLMs in Your Smart Invoice → ERP System

LLMs here are **not just general chatbots** — they're **specialized, narrow-context models** guided by **prompts, fine-tuning, or adapters**.

You won't be training a huge LLM from scratch (too heavy for M.Tech). Instead, you'll design them by **combining existing LLMs with domain-specific training and structured prompts**.

◆ Approaches for LLM Design

1. Prompt Engineering (Lightweight, Quick to Implement)

- Craft structured prompts with **role + schema + constraints**.
- Example for extraction agent:

You are an invoice parsing assistant.

Extract the following fields: Invoice_No, Vendor_ID, PO_No, Date, Amount.

Return output strictly in JSON format with confidence score for each field.

👉 Best when you want a **baseline system quickly**.

2. Fine-tuning / Domain Adaptation (For Precision)

- Take an open-source LLM (e.g., **LLaMA 3, Falcon, Mistral**).
- Fine-tune it on **invoice + ERP datasets** (from RVL-CDIP, FUNSD, synthetic invoices).
- Focus: extraction accuracy, validation reasoning.
- Example: train model on Invoice_Text → Extracted_JSON.

👉 Best for your **core extraction & validation agents**.

3. Retrieval-Augmented Generation (RAG)

- Store ERP master data (Vendors, PO numbers) in a **vector DB**.
- LLM queries the DB → improves **validation agent** decisions.
- Example:

- Input: “Vendor: Abc Corp”
- RAG → finds “Vendor_ID = V9821” in DB
- LLM outputs corrected JSON.

👉 Best for **cross-checking with ERP master data**.

4. Instruction-Tuned Mini LLMs (Agent Specialization)

- Instead of one big LLM, design **small LLMs per agent**:
 - **Extraction LLM** → text-to-JSON.
 - **Validation LLM** → rule reasoning.
 - **Exception LLM** → user Q&A.
 - **ERP LLM** → text-to-IDoc/XML transformation.

👉 Each LLM = “narrow specialist” with minimal context.
This matches perfectly with **MCP agent modularity**.

◆ How Each Agent Uses LLM

Agent	LLM Role	Design Approach
Data Extraction	Convert raw OCR text → structured fields	Fine-tuned LLM on invoice data + JSON-format prompt
Validation	Check extracted data against rules/DB	RAG (vector DB for vendor/PO lookup) + rule-based prompting
Exception Handling	Generate clarification Qs, interact with user	Prompt templates for error → question generation
ERP Integration	Transform JSON → ERP format (XML/IDoc/JSON)	Instruction-tuned prompt for schema transformation
Orchestrator (MCP)	Route tasks, summarize logs	Lightweight prompt-based LLM (no fine-tuning)

◆ Example LLM Design for Extraction Agent

Prompt:

You are a financial document parser.

Task: Extract Invoice Number, Vendor ID, PO Number, Date, and Amount.

Constraints:

- Output must be valid JSON.
- Include confidence scores (0-1).
- Do not invent missing fields.

Input (OCR text):

Invoice Number: INV-45678

Vendor: ABC Corp (ID: V9821)

PO: PO12345

Date: 10-09-2025

Amount: 225000 INR

Output:

```
{  
  "Invoice_No": {"value": "INV-45678", "confidence": 0.95},  
  "Vendor_ID": {"value": "V9821", "confidence": 0.92},  
  "PO_No": {"value": "PO12345", "confidence": 0.90},  
  "Date": {"value": "2025-09-10", "confidence": 0.96},  
  "Amount": {"value": "225000", "confidence": 0.98}  
}
```

◆ **Tech Stack for Designing LLMs**

- **Base Models:** LLaMA 3, Mistral, Falcon, GPT (API if allowed).
- **Fine-tuning Tools:** HuggingFace Transformers, PEFT/LoRA.
- **RAG:** LangChain, LlamaIndex, FAISS/Weaviate (vector DB).
- **Serving:** vLLM, FastAPI, Docker.
- **Evaluation:** BLEU/F1 for extraction, Accuracy for validation.

⚡ **In short:**

You don't need to *train from scratch*.

You'll **design LLMs by specialization** → each agent gets:

- Carefully engineered prompts
- Domain fine-tuning (if dataset available)
- RAG for ERP knowledge

Designing the AI Agents

Each agent = **(Specialized LLM + MCP wrapper + APIs)**.

They collaborate under **MCP (Model Context Protocol)** → ensuring modularity, orchestration, and traceability.

◆ 1. Data Extraction Agent

Goal: Extract structured fields from raw OCR invoice text.

- **Input:** OCR text (unstructured)
- **Process:**
 - Pre-process with NER/regex for hints
 - LLM converts → JSON schema
- **Output:** JSON (Invoice_No, Vendor_ID, PO_No, Date, Amount, Tax, Currency...)

Design Choices:

- LLM: Fine-tuned on invoice datasets (RVL-CDIP, FUNSD + synthetic data)
 - Prompting style: *"Extract fields, return strict JSON with confidence scores."*
 - Tools: HuggingFace + PEFT (LoRA)
-

◆ 2. Validation Agent

Goal: Verify extracted values against ERP rules + vendor/PO master data.

- **Input:** JSON (from Extraction Agent) + ERP DB context
- **Process:**
 - Cross-check Vendor_ID & PO_No with ERP DB (via RAG/vector DB)
 - Apply rules: date format, currency consistency, tax % range
- **Output:** Validated JSON with status: valid/invalid and error reasons

Design Choices:

- LLM: Prompt-based reasoning model (few-shot, with rules in system prompt)
 - RAG: ERP master data stored in FAISS/Weaviate
 - Example prompt:
"If PO_No not in DB, mark as Invalid and generate correction suggestions."
-

◆ 3. Exception Handling Agent

Goal: Handle errors/uncertainties by interacting with user.

- **Input:** Error JSON (e.g., Vendor_ID mismatch)
- **Process:**
 - LLM reformulates error → user-friendly clarification question
 - Accepts user input → converts back into structured JSON update
- **Output:** Corrected JSON

Design Choices:

- LLM: Instruction-tuned for Q&A style
 - Dialogue memory: LangChain conversational buffer
 - Example:
"Vendor ID V9821 not found. Did you mean V9827 (ABC Corp)?"
-

◆ 4. ERP Integration Agent

Goal: Push validated invoices into ERP (SAP/Oracle/MS Dynamics).

- **Input:** Validated JSON
- **Process:**
 - Map JSON → ERP schema (IDoc/XML/CSV)
 - Call ERP API / staging table
- **Output:** Confirmation or error log

Design Choices:

- LLM: Instruction-based transformer for **JSON → ERP XML mapping**
 - Tools: SAP BAPI/IDoc APIs or REST endpoints
 - Example:
 - {"Invoice_No": "INV-45678", "Vendor_ID": "V9821"}
 - → <INVOICE><ID>INV-45678</ID><VENDOR>V9821</VENDOR></INVOICE>
-

◆ 5. Audit & Logging Agent

Goal: Maintain full traceability for compliance.

- **Input:** Logs from all agents
- **Process:**
 - LLM summarizes anomalies, user interventions, processing time

- Generates compliance-ready reports
- **Output:** Audit JSON + human-readable summary

Design Choices:

- LLM: Small summarization model (distilBART, GPT-3.5-level)
- Example output:
"Invoice INV-45678 flagged for Vendor mismatch, corrected by user → posted successfully."

◆ **6. MCP Orchestrator Agent (Controller)**

Goal: Route tasks across all agents.

- **Input:** New invoice event
- **Process:**
 - Calls extraction → validation → exception handling → ERP integration
 - Tracks pipeline state
- **Output:** End-to-end workflow result

Design Choices:

- LLM: Lightweight controller (can even be rule-based, doesn't need training)
- Orchestration: LangGraph / MCP agent framework

🔑 **Design Summary Table**

Agent	LLM Design	Unique Features
Data Extraction	Fine-tuned LLM + JSON prompt	Domain adaptation on invoices
Validation	Prompt-based reasoning + RAG	ERP master data lookup
Exception Handling	Instruction-tuned Q&A model	Human-in-the-loop correction
ERP Integration	Prompt → schema transformer	Maps JSON → IDoc/XML
Audit & Logging	Summarization LLM	Compliance-ready reports
MCP Orchestrator	Lightweight LLM/rule-based	Routes tasks & manages context

